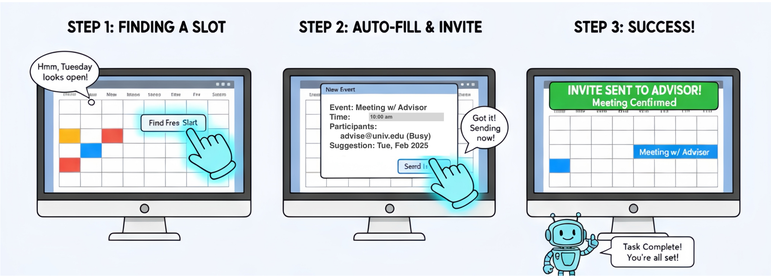
Inference-Time Scaling for GUI Agents with Process Reward Models and Internal World Models

Bella Wang yuw170@ucsd.edu
Rita Yujia Wu yuw172@ucsd.edu
Shuchang Liu shl153@ucsd.edu
Ziyu Huang zih029@ucsd.edu
Mentors: Kun Zhou, Zhiting Hu kuzhou@ucsd.edu zhh019@ucsd.edu

UC San Diego
HALICIOĞLU DATA SCIENCE INSTITUTE

Background & Introduction

GUI agents are systems that can use real software like a person. They look at the screen, click buttons, type into forms, and navigate multi-step workflows across browsers, operating systems, and mobile apps. Instead of building a custom integration for every app, a capable agent could automate many everyday tasks end-to-end.

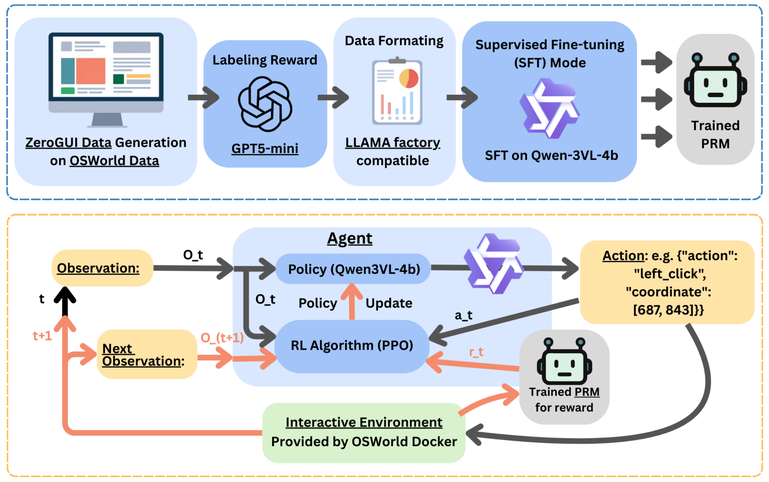


However, the hardest part is making these long sequences reliable. Therefore, in this project, we decided to approach it by letting the model take longer action sequences and consider more possibilities (inference-time scaling):

- **Training with a Process Reward Model (PRM):** teaches the policy what "good progress" looks like at each step, so longer rollouts stay goal-directed. (Like a coach saying "yes, keep going" / "no, that's a detour" during the learning process.)
- **Inference with an internal world model:** dynamically retrieves contrastive past experience to support *think-before-acting* reasoning and guide action selection. (Like recalling a similar situation on the spot.)

Method Part 1: Process Reward Model and Guided Agent Training

Our method is implemented according to the pipeline illustrated in the figure below.



1. **Inputs (per step):** task instruction (goal) + recent screenshots + the action the agent just took.
2. **PRM output:** a progress score (0/1 or scaled to [0,1]) and an optional short reason (for debugging only).
3. **How we fine-tune the PRM:** generate and collect OSWorld trajectories → label each step with progress/no-progress with GPT-5-mini → fine-tune a Qwen3-VL-4b model to predict the step score from the inputs above.
4. **How we train agent with PRM:** use Qwen3-VL-4B as the backbone policy model. During RL training, after each action, we query the PRM for a step reward and use PPO to optimize the policy. We train the model with LoRA for parameter-efficient fine-tuning, so the agent can learn from PRM feedback while staying goal-directed over longer rollouts.

PRM and Trained Agent Evaluation Result

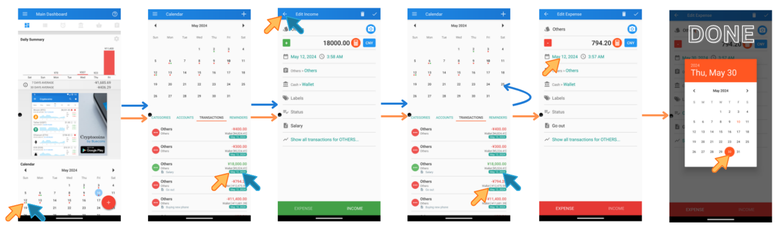
PRM evaluation on the OSWorld test set (505 three-step windows). The following are the confusion matrix counts and the corresponding derived metrics.

Model	TP	FP	FN	TN	Acc.	Prec.	Rec.	F1
Qwen3-VL-4B (zero-shot PRM)	111	9	304	81	38.02%	92.50%	26.75%	41.50%
Fine-tuned PRM (LLaMA-Factory)	313	41	102	49	71.68%	88.42%	75.42%	81.40%

We evaluate our trained agent and compare it with our baseline model on several benchmarks; the results are shown in the following table.

Model	UI-Vision	MMBench-GUI	AndroidControl High Step SR	Mind2Web Step SR	AndroidWorld	AndroidWorld Verification
Qwen3-VL-4B	**83.17%**	27.76%	31.64%	16.50%	**24.00%**	71.00%
Ours	82.92%	**28.13%**	**31.66%**	**17.50%**	**24.00%**	**86.00%**

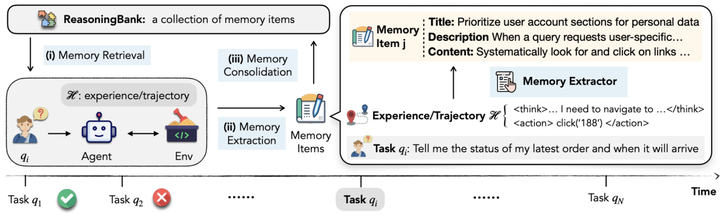
Agent Evaluation Case (Android Lab)



Method Part 2: Internal World Model

Goal. Improve long-horizon GUI reliability by enabling the agent to *think before acting*. The agent retrieves and contrasts past successful and failed trajectories to guide action selection and avoid repeating known failure patterns.

1. Experience Abstraction and Memory Storage



Agent interaction trajectories are converted into structured **memory items** and indexed using **FAISS**. We maintain two memory banks: 1) **Success trajectories**: effective behaviors; 2) **Failure trajectories**: common mistakes.

This contrastive memory allows the agent to retrieve both positive and negative evidence when reasoning about the next action.

2. State-Aware Contrastive Retrieval

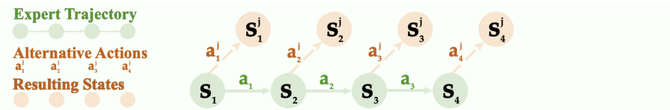
At inference time, the system retrieves trajectories conditioned on the **current screenshot and task** and performs contrastive analysis to guide the next action.

Current state (screenshot + task) → Retrieve top-k success + failure trajectories → Contrastive analysis → Step-level guidance

Contrastive analysis extracts:

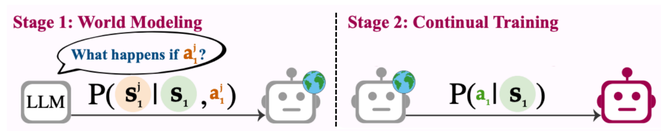
- **Success patterns** — action sequences leading to completion
- **Failure patterns** — behaviors associated with errors
- **Divergence points** — key decisions where trajectories differ

3. Candidate Action Reasoning



Using retrieved evidence, the agent evaluates the possible next actions relative to previously observed outcomes.
The world model prioritizes actions consistent with successful trajectories while discouraging behaviors associated with failures, enabling more deliberate decision-making during multi-step navigation.

4. Interpreting the System as a World Model



The contrastive memory system functions as an **implicit world model**. By comparing outcomes of similar past trajectories, the agent estimates which actions are more likely to lead to successful future states.
To improve robustness, we incorporate lightweight confidence checks and aggregate evidence across retrieved neighbors, stabilizing guidance when retrieval signals are weak.

World Model Evaluation

Setup. Experiments are conducted on the **WebVoyager** benchmark using the **Qwen/Qwen2.5-VL-7B-Instruct** vision–language GUI agent. The world model is built on the CoMEM memory framework with FAISS retrieval over CLIP embeddings.

Comparison. We compare a success-only retrieval baseline (original CoMEM) against our contrastive retrieval that retrieves both success + failure trajectories.

Result. Contrastive retrieval improves success rates on **3/4 domains**, with the largest gain on Amazon (+19.5 percentage points).

	Google Maps	Amazon	Allrecipes	Coursera
Baseline	16.67%	19.51%	15.91%	2.38%
World Model	26.83%	39.02%	11.11%	9.52%
Δ	+10.16	+19.51	-4.80	+7.14

Retrieval: top-k=3 success + 3 failure trajectories per step (FAISS + CLIP). *Dynamic retrieval:* ~40% of retrieved trajectories change between step 0 and step 5.

Conclusions

For PRM-Guided Agent Training, we built and curated datasets for PRM training, fine-tuned a PRM, and used it to provide dense step-wise rewards for GUI-agent training.

- **Limitation** Due to time and compute constraints, we trained the GUI agent for only 1 epoch and used only the first 4 steps of each rollout for PRM-guided agent training.
- **Planning Ability** Even under this limited training budget, PRM-based dense rewards led to improved performance on planning-related tasks.
- **Step Success** We also observed gains in step success rates (step SRs), suggesting that PRM supervision helps the agent make better local decisions during multi-step interaction.

Internal World Model. We introduce a contrastive memory mechanism that retrieves both successful and failed trajectories to guide agent actions during inference.

- **Contrastive retrieval memory:** Dual FAISS indices (success vs. failure) with CLIP embeddings enable retrieval of both positive and negative examples for decision guidance.
- **State-aware step guidance:** Retrieval is repeated at each step using the current screenshot, keeping guidance aligned with the evolving GUI state.
- **Empirical improvement:** Improves WebVoyager success rate on 3/4 domains, with the largest gain on Amazon.

[1] MathWorks. *Reinforcement Learning Agents*. MathWorks, 2026. Accessed: 2026-03-05.

[2] Wenyi Wu, Kun Zhou, Ruoxin Yuan, Vivian Yu, Stephen Wang, Zhiting Hu, and Biwei Huang. Auto-scaling continuous memory for gui agent. *arXiv preprint arXiv:2510.09038*, 2025.